

Chapter 10

Simulation of Computer Systems

Overview

10.1 Simulation Tools

10.2 High Level Computer-System Simulation

10.3 CPU Simulation

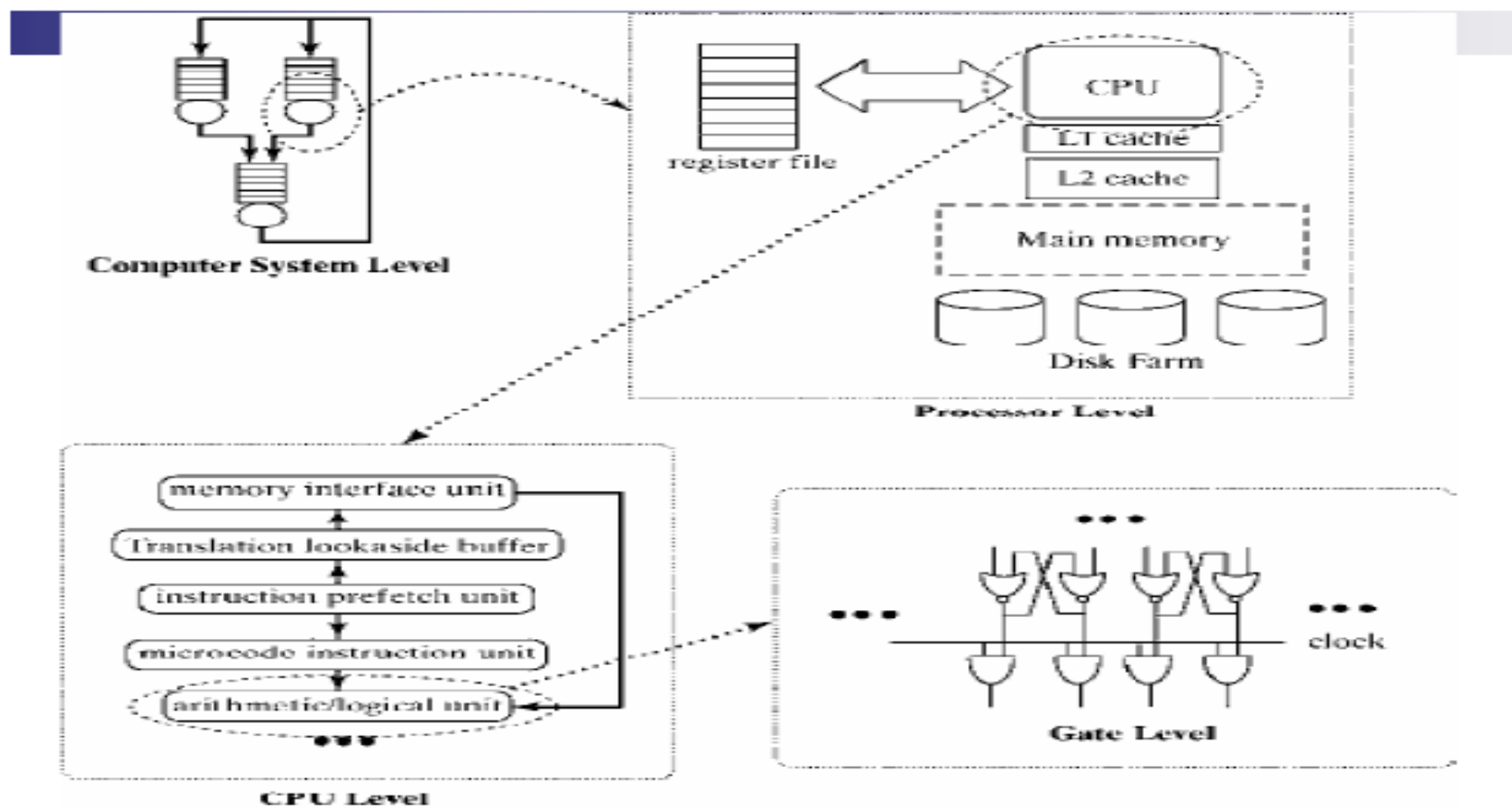
10.4 Memory Simulation

10.5 Simulation of computer networks

Introduction

- Computer systems are incredibly complex.
- A computer system exhibits complicated behavior at time scales from the time to “flip” a transistor’s state (on the order of 10^{-11} seconds) to the time it takes a human to interact with it (on the order of seconds or minutes).
- Computer systems are designed hierarchically, in an effort to manage the complexity.
- At a high level of hierarchy (the system level), one might view computational activity in terms of tasks circulating among servers, queuing for service when a server is busy.
- Beyond it, hierarchy can view activity as being among components of a processor (registers, memory hierarchy).
- At lower level, one views the activity of functional units that together make up a CPU and at even lower level, one can view the logical circuitry that makes it happen.

Different Level abstractions



Simulation Tools

- With different level of abstraction levels, one finds different tools used to perform and evaluate simulations.
- An important characteristics of a tool is how it helps in model building.
- Different simulation tools exist for each level

VHDL

- AT low level of abstraction
- Language used for describing digital electronic systems, that serves both as design specification and simulation specification.
- Design specification is matter of topology and simulation specification is matter of behavior.
- Modular design

Simulation Tools

- Some commercial simulation language do not support interaction with general programming languages.
- However simulation languages are not frequently used in the academic computer-science world.
- Cost is a partial explanation because commercial packages are developed with commercial needs and commercial budget in mind.
- Another explanation is the matter of emphasis: simulation languages tend to include a rich number of predefined simulation objects and actions and allow access to programming language to express object behavior.

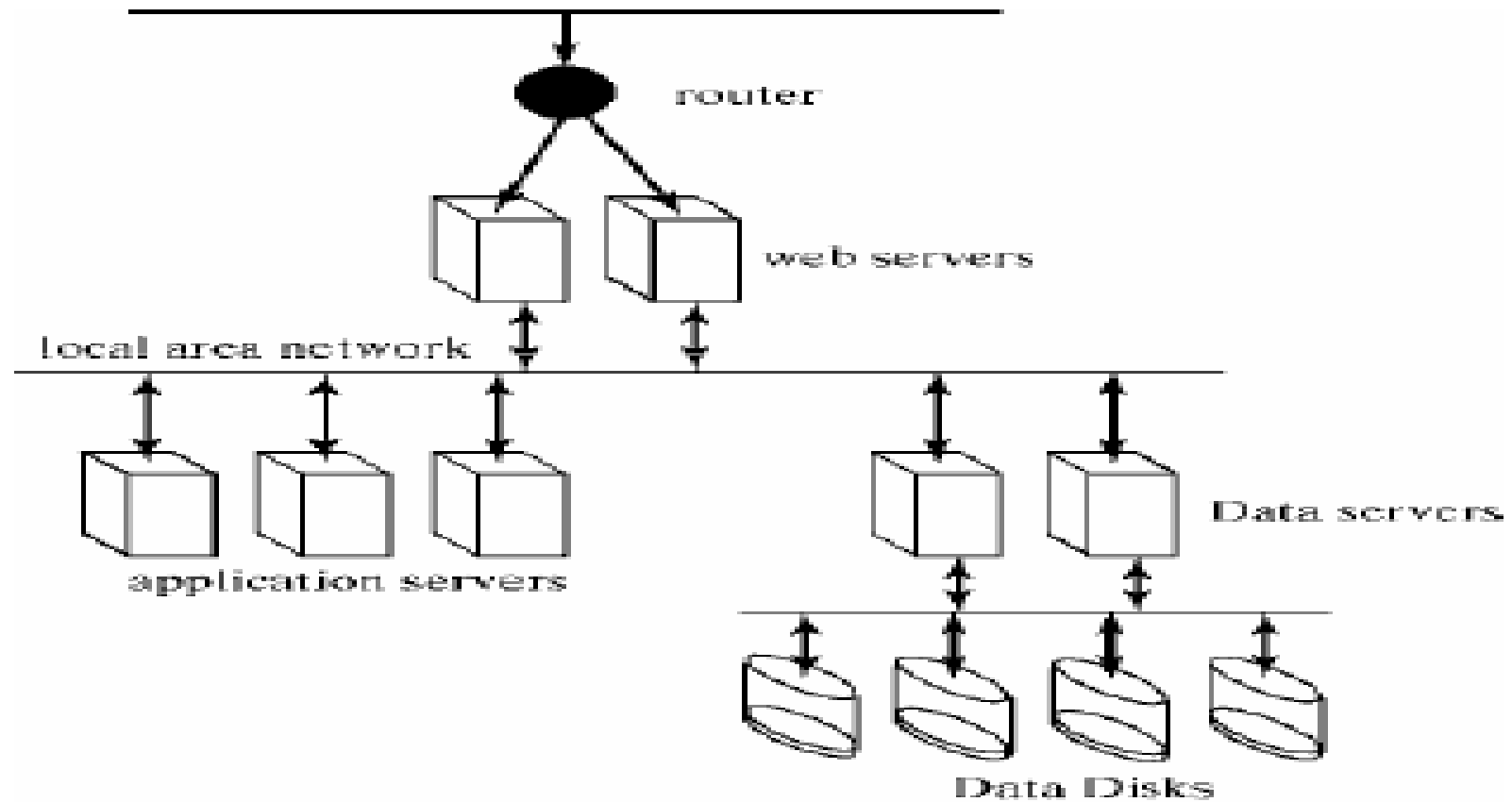
Simulation Tools

- A simulation model is expressed primarily in the constructs of the simulation language, and the model is evaluated either by compiling the model using a simulation-language-specific compiler and running it or by using a simulation-language-specific interpreter.
- Some tools have so much preprogrammed functionality that it is possible to design and run a model without writing a single line of computer code.
- By contrast, programming language with simulation constructs tend to define a few elemental simulation objects; a simulation model is expressed principally via the notions and control flow of the general programming language, with references to simulation objects interspersed.
- Among tools supporting user-programmed behavior, a fundamental characteristic is the worldview that is supported.

High Level Computer Simulation

- In this section, we illustrate the concept of high-level computer simulations sketching a simulation model of a computer system that services requests from the World Wide Web.
- We need to find the good response time.
- We should find bottleneck (delay).

Web site Server System



Web site Server System

- A company that provides a major website for searching and links to sites for travel, commerce, entertainment, and the like wishes to conduct a capacity-planning study.
- The overall architecture of it is shown above.
- At the back end, one finds data servers responsible for all aspects of handling specific queries and updating databases.
- Data servers receive requests for services from application servers-machine dedicated to run specific applications supported by the site.
- In front of the applications are web servers, which manage the interaction of applications with the world wide web.
- The portal to the whole system is a load-balancing router that distributes requests directed to the website among the web servers.

Web site Server System

- The goal of the study is to evaluate the site's ability to handle load at peak periods.
- Router have table of sessions
- Web server has three queues of threads
- Application server has two queues of threads
- Goal is to find response time distribution
- First we find bottleneck and then look how to reduce load at bottleneck during change of scheduling policy, bidding applications to servers, increasing CPU and I/O devices

Web site Server System parameters

Subsystem	Specification
Router	Load balancing policy, execution times
Web server	Server count, queuing policy, execution times
Application Server	Server count, queuing policy
Data Server	Server count, Disk count, queuing policy, Disk time

CPU Simulation

- We consider a lower level of abstraction and look at the simulation of a CPU. Whereas the high-level simulation of the previous example treated execution time of a program as constant, at the lower level we do the simulation to discover “what is execution time?”
- The input driving this simulation is streams of instructions.
- The simulation works through the mechanics of the CPU’s logical design to find out what happens in response to that stream, how long it takes to execute the program and where bottleneck exists in CPU design.
- Main challenge to make effective use of a CPU is to avoid stalling it; stalling happens whenever the CPU commits to executing an instruction whose inputs are not all ready or present.
- A leading cause of stalls is the latency delay between CPU and main memory, which can be tens of CPU cycles.

CPU Simulation

- E.g. one instruction might initiate a read: `load R2, 4(R3)` which is an assembly language statement that instructs the CPU to use the data in register 3 (after adding value 4 to it) as a memory address and to put the data found on that address into register 2.
- If the CPU insisted on waiting for that data to appear in a register 2 before further execution, the instruction could stall the CPU for a long time if the referenced address is not found in the cache.
- High performance CPU avoid it by recognizing additional instructions can be executed, up to the point where the CPU attempts to execute an instruction that reads the content of register 2 for example:

`add R4, R2, R5`

CPU Simulation

- This instruction adds the contents of registers 2 and 5 and places the result in register 4. if the data expected in register2 is not yet present, the CPU will stall.
- This sort of arrangement just described was first used in the earliest supercomputers. Modern microprocessors add some additional capabilities to exploit ILP (Instruction level parallelism)
- Pipelining has long been recognized as way of accelerating the execution of computer instructions.
- Pipelining exploits the fact that each instruction goes sequentially through several stages in the course of being processed; separate hardware resources are dedicated to each stage, permitting multiple instructions to be in various stages of processing concurrently.

ILP CPU

- A typical sequence of stages in an ILP CPU is as follows:
 - Instruction fetch - the instruction is fetched from the memory.
 - Instruction decode – the memory word holding the instruction is interpreted to discover what operation is specified; the register involved are identified.
 - Instruction issue – an instruction is “issued” when there are no constraints holding it back from being executed.
 - Instruction execute – the instruction is performed.
 - Instruction complete – results of the instruction are stored in the destination register.
 - Instruction graduate – executed instructions are graduated in the order that they appear in the instruction stream.

Simulation model of ILP CPU

- Fetch
 - Read from simulated memory and cache
- Decode
 - Register mapping
 - Branch prediction
- Issue
 - Input registers must be available
 - Functional units must be available
- Execute & complete
 - Find branch
 - Register writing
 - Release functional units and registers
- Graduation
 - Find exception

Memory Simulation

- One of the great challenges of computer architecture is finding way to deal effectively with the increasing gap in operation speed between CPU and memory.
- Solution is to use hierarchies of memories:
 - a relatively small memory- the L1 cache operates at CPU speed
 - a large memory the L2 cache is larger and operates more slowly
 - The main memory is larger still and even slower
 - The smaller memory hold data that was referenced recently and nearby data that one hopes will also be referenced soon.
 - Data moves up the hierarchy on demand and ages out as it becomes disused, to make room for the data in current use.

Memory Simulation

- The principle measure of the quality of a memory hierarchy is its hit ratio at each level, therefore one way to improve quality of memory hierarchy can be by increasing hit ratio by using following methods:
- Set associativity
 - It uses what is known as associative memory, one that associates search keys with data.
- Full associative
 - A full associative cache is one where any address can appear anywhere in the cache.
- Replacement Policy (Least Recently Used)
 - When a reference is made but is not found in a set, some block in the set is ejected to make room for the one containing the new reference. Under LRU, the block selected for ejection is the one which, among all blocks in the set, was last referenced most distantly in the past.

Simulation of computer networks

- Computer network simulation is the process of creating a virtual model of a computer network to study its behavior without building the actual network.
- It helps evaluate:
 - Network performance
 - Communication delays
 - Packet loss
 - Throughput
 - Congestion
 - Reliability

Why Simulate Computer Networks?

- Predict network performance before deployment.
- Compare different network protocols.
- Identify bottlenecks.
- Reduce implementation cost.
- Test large-scale networks safely.

Components of a Network Simulation Model

- **Nodes:** Computers, routers, switches, servers.
- **Links:** Wired or wireless communication channels.
- **Packets:** Units of transmitted data.
- **Traffic:** Data generated by users or applications.
- **Routing Protocol:** Determines packet paths.
- **Queues/Buffers:** Store packets waiting for transmission.

Popular Network Simulation Tools

- Cisco Packet Tracer
- NS-3
- GNS3
- OMNeT++
- Boson NetSim
- Mininet

Reference

- “Discrete Event system simulation”, 4th Edition, Jerry Banks, John S. Carson II, Barry L. Nelson, David M. Nicol, P. Shahabudeen